Motion Detection Cut

Prezentare intermediară - Milestone 1

Programare concurentă și distribuită

Echipă:

Damian Andrei
Nagy Robert
Groza Paul
Domocoș Darian

1 Introducere

1.1 Scop

Acest document descrie specificațiile unui sistem Client–Server pentru detectarea mișcării într-un fișier video. Sistemul utilizează procesare paralelă bazată pe procese (`fork()`) și urmărește identificarea cadrelor în care nu există mișcare.

1.2 Context

Aplicația este dezvoltată în C/C++ și integrează concepte din mai multe domenii:

- comunicație în rețea (sockets TCP)
- programare concurentă (procese multiple)
- procesare video (OpenCV)
- configurare externă (libconfig)

Sistemul este destinat utilizării academice pentru demonstrarea integrării acestor concepte.

1.3 Definiții

- **Motion Detection** – procesul de identificare a diferențelor între cadre consecutive
- **Worker** – proces copil care procesează un segment din video
- **Chunk** – subset de cadre atribuit unui worker
- **No-motion frame** – cadru în care nu se detectează mișcare

2 Descriere generală

2.1 Arhitectură

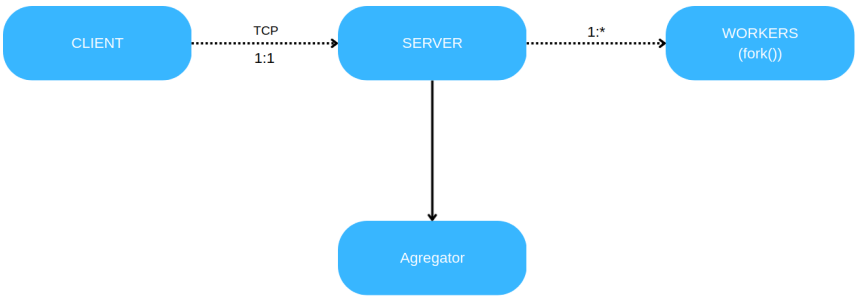

Figure 1: Diagrama proceselor în sistemul Client–Server

Sistemul este structurat pe model Client–Server, unde clientul trimite cereri de procesare, iar serverul coordonează execuția prin procese copil.

2.2 Flux execuție

1. Clientul trimite o cerere către server
2. Serverul validează cererea și parametrii
3. Video-ul este împărțit în segmente (chunk-uri)
4. Se creează procese copil folosind `fork()`
5. Fiecare worker procesează segmentul atribuit
6. Rezultatele sunt colectate de procesul părinte
7. Serverul transmite rezultatul final către client

3 Protocol și FCE

3.1 Mesaje

Protocolul utilizat este simplu, text-based, pentru a reduce complexitatea implementării.

Request:

```
REQ|video_path|chunk_size|threshold
```

Response:

```
RES|no_motion_frame_count
```

3.2 FCE

Files:

- fișier video de intrare
- fișier de configurare (`server.cfg`)

Communication:

- sockets TCP pentru client-server
- pipe() pentru comunicarea între procese

Execution:

- `fork()` pentru paralelizare
- `wait()` pentru sincronizare

4 Specificații funcționale

4.1 Client

Clientul este responsabil pentru:

- parsarea argumentelor din linia de comandă folosind `getopt`
- validarea datelor introduse de utilizator
- trimiterea cererii către server
- afișarea rezultatului primit

4.2 Server

Serverul gestionează întregul proces de execuție:

- citește parametrii din fișierul de configurare
- acceptă conexiuni TCP
- distribuie sarcina către procese copil
- colectează și agregă rezultatele

4.3 Worker

Fiecare worker:

- procesează un subset de cadre
- utilizează OpenCV pentru analiză
- determină dacă există mișcare
- transmite rezultatul către procesul părinte

5 Algoritm Motion Detection

Algoritmul utilizat este simplu și eficient:

```
frame_diff = abs(frame_t - frame_t-1)
if frame_diff < threshold -> no motion
```

Acest algoritm permite detectarea rapidă a diferențelor între cadre consecutive, fiind potrivit pentru un sistem demonstrativ.

6 Arhitectură detaliată

6.1 Structură procese

```
Parent (Server)
 |-- Worker 1
 |-- Worker 2
 |-- Worker N
```

Procesul părinte coordonează execuția și gestionează comunicarea, în timp ce procesele copil execută sarcini independente.

7 Config

Fișierul de configurare permite flexibilitate în execuție:

```
server:
{
    max_workers = 4;
    chunk_size = 100;
    threshold = 25;
};
```

8 Constrângeri

- nu se utilizează multithreading
- comunicația este realizată doar prin sockets și pipe-uri
- implementarea este simplificată pentru scopuri academice
- codul trebuie să fie compatibil cu `clang-tidy`

9 Concluzie

Sistemul demonstrează integrarea mai multor concepte importante:

- procesare paralelă
- comunicație în rețea
- procesare video

Implementarea este simplă, dar eficientă, fiind potrivită pentru scopuri educaționale.